

IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS

Project Proposal for Volunteership

Proponent: Sarthak Mushrif

Target Domain: Distributed Systems, Compiler Design, Cloud Computing

1 Executive Summary

This proposal outlines the development of **IsoShard Proxy**, a high-performance database middleware designed to enforce **ACID consistency** across horizontally sharded Relational Database clusters.

While standard cloud architectures (such as Microservices) excel at availability, they often struggle with strong consistency in distributed write operations. IsoShard Proxy solves this by implementing a **Two-Phase Commit (2PC)** protocol managed by a custom **Java Netty** server. The system parses incoming SQL using a custom-built compiler frontend (leveraging CS571 principles), optimizes the query via Abstract Syntax Tree (AST) analysis, and coordinates atomic commits across multiple **AWS RDS** instances.

This project aims to bridge the gap between **Programming Language Theory** (parsing/interpreting) and **Cloud-Native Systems Engineering**, providing a robust platform for researching distributed consensus algorithms.

2 Problem Statement

Scaling a monolithic database vertically has a hard ceiling. The industry standard solution is **Sharding**—splitting data across multiple physical nodes. However, sharding introduces two critical failures:

1. **Lost Atomicity:** If a transaction requires debiting User A (Shard 1) and crediting User B (Shard 2), a network failure in the middle can result in data corruption (Partial Commit).
2. **Network Inefficiency:** Naive middleware fetches all data to the application layer to join it, causing massive I/O bottlenecks (known as "The N+1 Query Problem").

IsoShard Proxy addresses these by moving the transaction logic and query optimization *into* the network layer, ensuring that distributed transactions either fully succeed or fully fail (Atomic), with zero data corruption.

3 Technical Architecture

The system is architected as a multi-tier distributed application deployed entirely on **AWS**.

3.1 Layer 1: High-Throughput Network Protocol (Java Netty)

Instead of using HTTP/REST, which incurs significant text-processing overhead, the proxy utilizes a **Custom Binary Protocol** built on **Java Netty (NIO)**.

- **Reactor Pattern:** A single EventLoop thread manages thousands of concurrent connections using non-blocking I/O selectors, allowing the system to handle high-frequency workloads on minimal hardware.
- **Zero-Copy Networking:** Packet payloads are forwarded from Client → Proxy → Database without unnecessary memory copying in the JVM user space.

3.2 Layer 2: The SQL Compiler & Optimizer (CS571 Connection)

The core intelligence of the proxy is derived from Compiler Design principles.

- **Lexical Analysis & Parsing:** Using **ANTLR4**, the system tokenizes raw SQL queries and constructs an **Abstract Syntax Tree (AST)**.
- **Algebraic Rewriting:** A Visitor Pattern traverses the AST to identify optimization opportunities. For example, *Predicate Pushdown* detects WHERE clauses and routes them to the specific shard holding that data, minimizing network scan overhead.

3.3 Layer 3: Distributed Transaction Coordinator (The "CP" Mode)

To guarantee data consistency (Consistency over Availability in CAP theorem), the system implements a **Two-Phase Commit (2PC)** engine.

- **Phase 1 (Voting):** The Coordinator requests a "Prepare" lock from all involved AWS RDS shards.
- **Phase 2 (Commit):** If and only if *all* shards vote "YES," the Coordinator commits the transaction.
- **Write-Ahead Logging (WAL):** To survive system crashes, the Coordinator logs every state change (INIT, PREPARED, COMMITTED) to **AWS DynamoDB**. If the proxy restarts, it replays this log to recover system state.

4 Infrastructure & Technologies (AWS Native)

Component	Technology	Purpose
Compute	AWS EC2	Hosting the IsoShard Proxy middleware (Java 17) in an Auto-Scaling Group.
Data Shards	AWS RDS	Three distinct PostgreSQL instances simulating physical partitions.
Consistency Log	AWS DynamoDB	High-speed, low-latency storage for the Write-Ahead Log (WAL).
Compiler Language	ANTLR4 Java (JDK 17)	Generating the Lexer and Parser for the SQL dialect. Utilizing <code>CompletableFuture</code> for asynchronous 2PC protocol.

Table 1: System Technology Stack

5 Value Proposition

- **For the Department:** This project serves as a tangible demonstration of how theoretical concepts (Grammars, Finite Automata, ASTs) are applied in modern Cloud Engineering.

- **For Research:** The modular design allows us to swap the consensus algorithm (e.g., replacing 2PC with **Paxos** or **Raft**) to measure performance trade-offs, supporting potential research into distributed consistency models.

6 Conclusion

IsoShard Proxy is a rigorous engineering challenge that requires deep knowledge of **Database Internals**, **Compiler Theory**, and **Cloud Architecture**. By building a system that ensures transactional integrity in a chaotic distributed environment, this project demonstrates the advanced technical capability required for the Research Volunteership.
